언어 선택 Google 번역에서 제공

개발언어 C++

MFC 메모리 릭 체크시 유용한 방법 2개

10년 전 *by* 기억보다기록

MFC를 사용하여 프로그램을 개발하고 있다면,

```
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
```

위의 코드를 .cpp 화일에 넣어 프로그램이 종료 되었을때 아래와 같이 누수된 메모리를 할당하는 부분의 소스 코드와 라인 수를 출력 해준다.

```
Detected memory leaks!
Dumping objects ->
d:₩sample₩sample.cpp(35) : {48} client block at 0x003739D0, subtype 0, 4 bytes long.
Data: <( > 28 00 00 00
d:₩sample₩sample.cpp(34) : {47} client block at 0x00373990, subtype 0, 4 bytes long.
Data: < > 1E 00 00 00
Object dump complete.
```

위의 예에서는 두 블록의 메모리 누수가 검출 되었는데 각 라인의 의미는

d:₩sample₩sample.cp (35) : {48} client block at 0x003739D0, subtype 0, 4 bytes long.

sample.cpp 화일의 **35번째 라인**에서 할당된 메모리가 누수되었고 그것은 **48번째로 할당된 메모리**이며 **메모리 주소는 0x003739d0**, **4Byte가 누수**되었다는 의미로 해석할 수 있다.

그 경우엔 위의 DEFINE 문과 함께 아래와 같은 라인을 헤더에 삽입해서 해결할 수 있다.

```
#if !defined(_AFXDLL)
#include <windows.h>
#include <crtdbg.h>
#if defined(DEBUG) | defined(_DEBUG)
#if !defined(DEBUG_NEW)
#define DEBUG_NEW new(_CLIENT_BLOCK, __FILE__, __LINE__)
#endif
#endif
#endif
```

DEBUG_NEW를 새로이 정의함으로써 DEBUG_NEW를 찾을 수 없는 문제를 해결한다.

간혹 malloc 을 이용한 메모리 할당은 위의 내용이 적용이 되질 않는데 이 경우엔 아래의 문장을 소스코드에 삽입한다.

- 헤더 화일에는 아래라인을 추가

#define DEBUG_MALLOC(size) _malloc_dbg(size, _NORMAL_BLOCK, __FILE__ , __LINE__)

- malloc을 재정의 하려는 소스 코드에 아래 라인을 추가.

#define malloc DEBUG_MALLOC

위의 방법들로 memory leak dump 에서 할당된 소스 코드와 라인수를 바로 찾을 수 있다.

MFC로 프로젝트 만들어서 디버그하면 프로세스가 종료될 때 메모리 릭을 체크해서 출력창에 보여주는데, 일반 콘솔 어플리케이션이나 WIN32 어플리케이션 프로젝트로 작업하면 그런 정보를 알려주지 않아서 가끔 아쉬울 때가 있다.

그런데.. 이 '메모리 릭 검출 기능'이 MFC 프로젝트에서만 가능한 게 아니라 함수 하나만 호출해 주면 다른 프로젝트에서도 가능하다는 걸 알았다.

그냥 간단하게,

```
#include <crtdbg.h>

_CrtSetDbgFlag( _CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF );
```

이렇게만 추가해 주면 된다.

(함수 호출은 main이나 WinMain함수 안에서...)

CRT(C Runtime library)를 사용합시다.

아래의 코드를 사용하면 메모리 릭(메모리 누수, Memory Leak)에 효과적으로 대처할 수 있다.

주의할 점은 사용법을 명확히 숙지하고서 사용할 것.

Introduce.

```
int WINAPI wWinMain( HINSTANCE hInstance, HINSTANCE hPrevInstance, LPWSTR lpCmdLine, int nCmdShow )
{
// Enable run-time memory check for debug builds.
#if defined(DEBUG) | defined(_DEBUG)
_CrtSetDbgFlag( _CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF );
#endif
```

다이렉트 X예제를 살펴보면 wWinMain시작 부분에 위와 같은 코드를 만날 수 있다.

이 코드는 디버그 시에 메모리릭이 발생하면 출력창에 몇 번지에서 얼마 크기의 메모리가 해제되지 않았는지를 표시 해 준다.

또한 메모리릭을 체크하기 위해서 <crtdbg.h>를 인클루드 해 줘야 하는데 DXUT.H를 인클루드 했다면 내부에서 <crtdbg.h>를

인클루드 해 주므로 따로 해 줄 필요가 없다.

아래에서 사용하는 코드들은

```
#if defined(DEBUG) | defined(_DEBUG)
```

로 감싸져 있기 때문에 릴리즈 모드에서는 동작하지 않는다. 또한 릴리즈 모드에서는 인클루드 자체가 제외 되기 때문에 효율을 걱정할 필요가 없다.

Step 1.

```
#if defined(DEBUG) || defined(_DEBUG)
#include <crtdbg.h>
#define new new(_CLIENT_BLOCK, __FILE__, __LINE__)

_CrtSetDbgFlag( _CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF );

#endif
```

필요한 헤더를 인클루드 해 주고

```
#define new new(_CLIENT_BLOCK, __FILE__, __LINE__)
```

new자체를 새로 정의 해 버렸다. 메모리 릭 보고를 해 줄 때 릭을 발생시킨 녀석의 파일 이름과 라인을 출력 해 주어서 디버깅 시에 출력창에서 바로 찾아볼 수 있다. __FILE__과 __LINE__는 미리 정의된 매크로로 가끔 사용해 주면 유용하다.

```
_CrtSetDbgFlag( _CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF );
```

이 녀석이 실제적으로 출력창에 메모리 릭 보고를 해 준다. 5가지의 Flag가 있다.

_CRTDBG_ALLOC_MEM_DF

디폴트로 켜져 있으며, 디버그 버전에서 모든 메모리 할당이 일어날 때마다 추적 가능하도록 특별한 기능을 추가해 둔다. 이 플랙이 켜져 있어야 메모리 누수를 안전하게 검사 할 수 있다.

_CRTDBG_DELAY_FREE_MEM_DF

delete, free등으로 삭제되어도 바로 삭제되지 않고, CRT의 메모리 관리소에 남아 있다가 프로그램 종료시에 완전히 삭제된다.

_CRTDBG_CHECK_ALWAYS_DF

CRT가 내부적으로 할당한 블록도 메모리를 체크할 때 포함한다. 일반적으로는 CRT가 할당한 블록은 메모리 체크에서 제외된다. 일반적으로 사용하지 않는다

_CRTDBG_LEAK_CHECK_DF

프로그램이 완전히 종료되기 직전에 아직 해제되지 않은 메모리가 있는지 검사한다. 프로그램의 종료 포인트가 여러군데 있는 경우에 사용하면 일일이 _CrtDumpMemoryLeaks 메소드를 호출하지 않아도 자동적으로 메모리 누수를 검사할 수 있게된다.

Step 2.

메모리 할당이 시작되기 이전에 최상위에 만들어 두자.

Step 3.

Detected memory leaks!

Dumping objects ->

{840} normal block at 0x0109E460, 320 bytes long.

Data: < _ > 9C 07 5F 00 CD CD CD CD CD CD CD CD CD CD CD CD

{838} normal block at 0x0109E398, 46 bytes long.

Data: <1 m B o x A > 31 00 6D 00 20 00 42 00 6F 00 78 00 20 00 41 00

{837} normal block at 0x0109E258, 260 bytes long.

Data: <testAlpha.tga > 74 65 73 74 41 6C 70 68 61 2E 74 67 61 00 CD CD

{836} normal block at 0x0109E218, 4 bytes long.

Data: < e > C0 65 1C 00

{835} normal block at 0x0109E198, 68 bytes long.

Data: < ? ? ? ?> 00 00 80 3F 00 00 80 3F 00 00 80 3F 00 00 80 3F

{834} normal block at 0x0109E128, 48 bytes long.

Data: < _ SA `W > A8 09 5F 00 98 E3 09 01 18 53 41 01 60 57 1C 00

Object dump complete.

D3DX: MEMORY LEAKS DETECTED: 3 allocations unfreed (708 bytes)

D3DX: Set HKLM₩Software₩Microsoft₩Direct3D₩D3DXBreakOnAllocId=0x21b to debug

고의적인 릭 발생이다.

메모리릭은 첫번째 할당된 순서 그리고 주소 크기를 보여주고

메모리 안에 위치한 데이터를 보여준다.

DXUT를 사용하지 않은 프로젝트에서는 new를 매크로로 씌워 놓았을 때 출력창에서 릭이 발생한 파일과 줄 번호가 표시되어 이를 더블클릭하면 그 위치를 직접 보여준다.

Step 4.

_CrtSetBreakAlloc(Number);

할당된 순서를 알 수 있다면 다음을 추가 시켜주자. 지정된 메모리가 할당될 때 디버깅을 중지하고 제어권을 디버거에게 넘긴다. 즉 어떤 놈이 할당을 시도하고 무책임하게 떠났는지 알 수 있다.

_CrtCheckMemory()

이 함수가 호출된 시점에서 지금까지 동적 할당된 모든 메모리를 체크해서 문제가 발생한 녀석이 있으면 0을 리턴한다.

0xFD : 메모리 블록 양 끝의 버퍼에 생성된다

0xCD : 새로 생성된 버퍼에 저장되는 기본값이다

0xCC : 스택에 변수가 생성되면 우선 이값으로 채워진다

0xDD : 삭제된 메모리 블록은 이 값으로 채워진다

P.S.

DirectX프로그래밍이 아닐때도 위 내용은 유용하게 사용할 수 있지만 만약 DirectX프로그래밍을 하고 있다면 추가적인 팁이 있다.

DirectX의 경우에는 COM기반의 프로그래밍 언어인데 인터페이스를 얻어온 후 Release를 하지 않았거나 혹은 AddRef()로 늘어난 래퍼런스 카운터에 맞게 Release()가 호출이 되지 않았다면 릭이 발생할 수 있다.

간혹 이런 경우 위의 함수들이 메모리릭을 일으킨 정확한 시점을 감지하지 못하는 경우가 생긴다. 그 때에는 D3DX에서 출력창에 띄워준 정보를 바탕으로 메모리릭을 찾을 수 있다.

위에서 봤던 출력창 메시지의 제일 마지막 2줄이다.

아래와 같은 순서로 메모리릭을 일으킨 인터페이스가 생성되는 시점을 찾을 수 있다.

1. 시작프로그램의 실행에서 REGEDIT를 실행해서 레지스트리 편집기를 실행하자.

2. HKEY_LOCAL_MACHINE -> SOFTWARE -> Microsoft -> Direct3D 의 위치로 이동

(설명된 위치 이외의 키들을 건드릴 경우 시스템에 치명적인 손상을 줄 수 있음)

3. D3DXBreakOnAllocId 라는 키가 존재하는 지 확인 하고 없을 경우에는

새로만들기 -> DWORD값 으로 새 키를 만들어 주고 이름을 위와 같이 설정 해 주자.

(비슷한 이름의 키가 존재할 수 있으니 이름이 정확히 맞는지 확인할 것 !)

4. 값에 16진수로 0x21b값을 적어주자.

(이 값은 출력창에 표시된 위의 값이다. 0x는 빼고 적어주자.)

5. 이제 디버그 모드로 실행을 시키면 메모리릭을 일으킨 범인이 할당되는 순간

브레이크 포인트를 걸고 제어권을 사용자에게 넘겨준다.

.cpp 파일 윗부분에

?

```
#ifdef _DEBUG

#undef THIS_FILE

static char THIS_FILE[] = __FILE__;

```

```
8

9    #endif
```

넣고 디버깅모드로 실행한뒤 프로그램 수행 후 종료하면

f:₩dd₩vctools₩vc7libs₩ship₩atlmfc₩src₩mfc₩strcore.cpp(156) : {7535} normal block at 0x02D7C310, 24 bytes long. Data: < Q > E4 B8 04 51 03 00 00 00 03 00 00 00 01 00 00 00

이런식으로 뜹니다.

일반적으론 new 후 delete를 안하거나 new []로 할당하고 delete 로 삭제해서 뜨는 경우가 대부분인데

이러한 경우는 발생한 위치를 정확하게 찾아줍니다.

위와 같은 경우엔 잘못된 Type_casting에 의한 경우일 수도 있고 그 외의 경우일 수도 있는데

중요한 것은 릭이 발생한 위치가 명확하지 않습니다.

하지만 프로그램의 수행 순서가 동일하다면 위의 표현에서 {7535} 가 메모리가 할당된 블럭을 나타냅니다.

이럴 경우 프로그램의 시작지점에

?

```
1    _CrtSetBreakAlloc( {사이의 숫자} );
```

이런식으로 세팅을 해주면 디버깅 모드로 동작 중 해당하는 메모리블럭에 할당을 시도할 때, Visual Studio가 알려줍니다.

따라서 그 부분의 잘못된 점을 수정하면 Memory Leak을 해결할 수 있습니다.

추가

C언어에서 사용하는 malloc calloc 같은 메모리 할당 함수도 Memory Leak을 발생시키고 추적할 수 있습니다.

위와 같이 .c 파일의 윗부분에

```
#ifdef _DEBUG
#include <crtdbg.h>
            #ifdef malloc
            #undef malloc
            #endif

            #define malloc (s) (_malloc_dbg(s, _NORMAL_BLOCK, __FILE__, __LINE__ ))
            #ifdef calloc
            #undef calloc
            #endif

             #define calloc(c, s) (_calloc_dbg(c, s, _NORMAL_BLOCK, __FILE__, __LINE__ ))
            #ifdef realloc
            #undef realloc
            #endif

            #define realloc(p, s) (_realloc_dbg(p, s, _NORMAL_BLOCK, __FILE__, __LINE__ ))
            #ifdef _expand
            #undef _expand

```

```
23
24          #define _expand(p, s) (_expand_dbg(p, s, _NORMAL_BLOCK, __FILE__, __LINE__
    ))
25
            #ifdef free
26
            #undef free
27
            #endif
28
29
            #define free(p) (_free_dbg(p, _NORMAL_BLOCK))
30
            #ifdef _msize
31
            #undef _msize
32
            #endif
33
34
            #define _msize(p) (_msize_dbg(p, _NORMAL_BLOCK))
    #endif
    </crtdbg.h>
```

공감 구독하기

TAG C++, DEBUG, MFC, visualstudio

개발언어/c++ 카테고리의 다른글

예외처리(Exception)

메모리 누수 관련 추적하여 LOG로 남기기

Rich Edit 사용법 메모

댓글 0

개발자로 살기

기억은 소중한 것 이지만 허무하게도 지워진 지식과 기억은 그토록 치열했던 삶의 흔적마저....

구독하기

댓글을 남겨주세요

**40세 미만이라면 이 매혹적
하지 마세요**

경복의 바다

COPYRIGHT 개발자로 살기, ALL RIGHT RESERVED.